Arabic Speech-to-Text Model Using wav2vec 2.0

Mahmoud Haytham and Mostafa AbdelAtty Sultan

December 23, 2024

Abstract

This report outlines the development of an Arabic speech-to-text (STT) system using the wav2vec 2.0 framework. The project involves extensive data collection, meticulous preprocessing, model training, detailed evaluation, and deployment via an API. This document provides a comprehensive overview of the project's steps, challenges faced, key findings, and suggestions for future work.

Contents

1 Introduction **2**

2 Data Collection and Preprocessing **2**
- 2.1 Datasets Overview . . . 2
- 2.2 Mozilla Common Voice Arabic Corpus . . . 2
- 2.3 Data Cleaning and Preprocessing . . . 3
- 2.4 Challenges in Data Handling . . . 3
- 2.5 Implementation Details . . . 3

3 Model Training **3**
- 3.1 Framework and Setup . . . 3
- 3.2 Challenges . . . 3
- 3.3 Training Results . . . 4

4 API Deployment and Testing **4**

5 Conclusion **5**

1 Introduction

Arabic is one of the most widely spoken languages in the world, characterized by significant diversity in dialects and accents across regions. This linguistic diversity, combined with limited resources for many Arabic dialects, poses unique challenges for developing accurate automatic speech recognition (ASR) systems. The primary objective of this project is to address these challenges by utilizing the wav2vec 2.0 framework, a state-of-the-art model for ASR tasks. This project focuses on Standard Arabic and several regional dialects, aiming to create a robust speech-to-text (STT) system capable of handling diverse speech data.

2 Data Collection and Preprocessing

2.1 Datasets Overview

To ensure the dataset was comprehensive and representative of Arabic's linguistic diversity, data was sourced from various domains and mediums. This included:

- **Movies**: Dialogues were extracted from a range of films to capture natural conversational speech.
- **Podcasts**: Diverse topics were included, with a notable example being *Ta Marbouta*, a podcast focusing on female speakers to ensure gender inclusivity and balance.
- **YouTube Videos**: Content was collected from various regions, such as the Levant, Gulf countries, and Egypt, to encompass dialectal variations.
- **Street Interviews**: Spontaneous speech captured in noisy, real-world environments to ensure robustness against background noise.
- **Radio Interviews**: Semi-structured conversations to complement other types of speech data.

The topics included news, entertainment, education, cultural discussions, and informal conversations, ensuring that the dataset was diverse, informative, and contextually rich.

2.2 Mozilla Common Voice Arabic Corpus

The Mozilla Common Voice Arabic corpus played a critical role in this project. It is an open-source dataset containing hours of transcribed audio in Standard Arabic and various dialects. The corpus was particularly valuable for:

- **Dialectal Coverage**: Ensuring representation of different regional accents.
- **High-Quality Annotations**: Providing manually verified transcriptions for training and validation.
- **Supplementing Proprietary Data**: Bridging gaps in linguistic diversity within the collected datasets.

This corpus served as a foundation for fine-tuning the model, enhancing its ability to generalize across dialects and contexts.

2.3 Data Cleaning and Preprocessing

Preparing the data for training involved multiple critical steps:

- **Silence Removal and Normalization**: Audio segments containing music were separated from speech, enhancing clarity and ensuring focus on linguistic features.
- **Transcription Cleaning**: Special characters, diacritics, and non-standard Arabic letters were removed or standardized. The script normalization ensured consistency across various dialects and writing styles.
- **Duration Management**: The lengths of audio files were optimized to ensure compatibility with memory and runtime limitations of the model training process.

2.4 Challenges in Data Handling

While constructing the dataset, the following challenges were encountered:

- **Imbalanced Data**: Certain dialects and genders were underrepresented, necessitating additional data collection efforts.
- **Inconsistent Annotations**: Variability in transcription quality, especially in crowd-sourced datasets like Mozilla Common Voice, required extensive manual correction.
- **Noisy Environments**: Data from street interviews and YouTube included significant background noise, making preprocessing more complex.
- **Ethical Considerations**: Ensuring proper licensing and fair use compliance while collecting public audio content.

2.5 Implementation Details

For this project, the `facebook/wav2vec2-large` model was chosen due to its superior performance in handling large-scale ASR tasks. Unlike the base model, the large version provides greater capacity and enhanced feature extraction, making it particularly suitable for the complex and varied nature of Arabic speech data.

3 Model Training

3.1 Framework and Setup

The wav2vec 2.0 model, initialized with the `facebook/wav2vec2-large` checkpoint, was fine-tuned on the collected Arabic datasets. Its ability to leverage self-supervised learning on raw audio inputs significantly reduced the dependency on large annotated datasets.

3.2 Challenges

Several challenges arose during the project:

- **Data Collection**: Ensuring compliance with fair use policies while collecting data from YouTube and other platforms required extensive manual effort.

- **Dataset Quality**: Some segments in the Mozilla Common Voice corpus included noisy or inconsistent transcriptions, requiring additional cleaning steps.
- **Storage Constraints**: The large size of WAV files made storage management on Kaggle and Google Colab challenging. A pipeline was devised to download, preprocess, and delete files sequentially.
- **Memory Issues**: Long-duration audio files caused memory overflows during training, requiring adjustments to the architecture and preprocessing pipeline to manage resource utilization effectively.
- **Runtime Interruptions**: Training on cloud platforms like Google Colab often led to interruptions. To mitigate this, checkpoints were saved at the best loss values, ensuring minimal progress loss in case of crashes.

3.3 Training Results

The following results were observed during training:

- **Epoch 1:** Training Loss = 2.49, Validation Loss = 2.24, WER = 0.78, CER = 0.97.
- **Epoch 2:** Training Loss = 0.99, Validation Loss = 16.97, WER = 0.64, CER = 0.96.
- **Epoch 3:** Training Loss = 0.80, Validation Loss = 19.03, WER = 0.60, CER = 0.96.
- **Epoch 4:** Training Loss = 0.71, Validation Loss = 22.96, WER = 0.57, CER = 0.96.

4 API Deployment and Testing

The trained model was deployed using FastAPI to provide a user-friendly API for speech-to-text transcription. The API allowed users to upload audio files and receive transcriptions in real time. The deployment process involved:

- Creating endpoints for file upload and processing.
- Integrating the trained wav2vec 2.0 model with the FastAPI backend.
- Testing the API with diverse audio samples.

While the transcription results from the model were accurate, several challenges were encountered during deployment:

- **Environment Issues**: Dependency conflicts in the Python environment led to errors during runtime, preventing seamless API functionality.
- **Resource Constraints**: The computational demand of real-time inference limited the API's performance on resource-constrained systems.

Despite these issues, the successful integration of the model demonstrated its potential for real-world applications.

5 Conclusion

This project successfully developed an Arabic STT system using the wav2vec 2.0 framework. The comprehensive dataset, robust preprocessing pipeline, and effective training strategies resulted in a model capable of handling diverse Arabic dialects. Future work could focus on:

- Expanding the dataset to include more dialects and topics.
- Addressing challenges in noisy environments through advanced noise-robust techniques.
- Resolving deployment challenges to ensure seamless API functionality.
- Optimizing the model for deployment on resource-constrained devices.